

Ryan Stoddard

Mar. 3rd, 2026

Foundations of Python Programming

Assignment 06

<https://github.com/RyanStoddard/IntroToProg-Python-Mod06>

SoC, Classes, and Functions

Introduction

In this document I will describe the steps I took to complete Assignment 06. This assignment asks us to create a python program that displays a menu. Depending on the user's choice within the menu they will be allowed to perform different tasks such as inputting data, displaying data, and writing data to a .json file. This exercise is nearly identical to assignment 5 except that it introduces classes, functions, and the "separation of concerns" pattern.

Separation of Concerns

One of the main differences in the code this time around is how it is organized. We will be separating our code into three main parts, processing, presentation, and logic. We still have the normal stuff at the start, such as a script header, imported modules, global constants and global variables. But instead of jumping right into the main body of the code we are going to create a FileProcessor class and fill it with any functions we need to read or write data to an external file. Then we will create an IO class fill it with any functions that interface with the user. Then we will finally jump into the main body of the code which is our "logic" part.

There are several advantages to writing code in this way. For one thing, it makes writing the logic part of the code much more straightforward. The code is much more streamlined since it isn't trying to juggle every possible aspect of making the code work as each new situation occurs. In fact, if you named your functions well that portion of the code almost reads like English. I find it much easier to follow when I'm not having to count indents and read past all the try/except verbiage masking the flow of the code.

Secondly, this separation of functionality makes making changes or improvements to how something works much easier to implement. For example, instead of having to hunt through every line of code for each instance I printed the names and courses to the user, I can just find the function responsible for that and fix it once. This lends itself well to future scaling of the program or possibly having multiple people working on different parts of the code while not interfering with each other's work.

Creating the "FileProcessor" Class and its Functions/Methods

We start our processing section of the code with the creation of our first class, named "FileProcessor". In order to do this, we simply type class FileProcessor: followed by a string literal section that acts as a tooltip/documentation for what the class is for. Any indented functions we create after this section will be considered part of the "FileProcessor" class. The two functions we'll create are the read_data_from

file function, and the write_data_to_file function. Both of these will be using a pair of parameters, one for file_name: str, and the other for student_data: list. Parameters are a way to locally contain variables so that they don't necessarily need to be a global variable. The code itself looks very similar to what we used in the last assignment. One difference is we must be careful to replace any global variables with the new parameter names. For example, "students" becomes "student_data", "FILE_NAME" become "file_name". I also utilized the IO.output_error_messages function within this function to handle all the error messages (that code appears later on, but still before the main body). I was also careful to include the "indent=2" this time during the json.dump section of writing data. (Figure 1).

```
57     @staticmethod 1 usage
58     def write_data_to_file(file_name: str, student_data: list):
59         """ This function writes data to an external json file
60
61         ChangeLog: (Who, When, What)
62         Ryan Stoddard,03/03/2026,Created function
63         """
64         file = None
65
66         try:
67             file = open(file_name, "w")
68             json.dump(student_data, file, indent=2)
69         except TypeError as e:
70             IO.output_error_messages(message: "Please check that the data is a valid JSON format", e)
71         except Exception as e:
72             IO.output_error_messages(message: "There was a non-specific error!", e)
73         finally:
74             if file is not None and file.closed == False:
75                 file.close()
76
```

Figure 1: Example of write_data_to_file Function

Creating the "IO" and its Functions/Methods

The first function created in the IO class is output_error_messages. This function is actually utilized in four other functions in this program. Which makes them much easier to write and read (Figure 2).

```
91     @staticmethod 7 usages
92     def output_error_messages(message: str, error: Exception = None):
93         """ This function displays a custom error messages to the user
94
95         ChangeLog: (Who, When, What)
96         Ryan Stoddard,03/03/2026,Created function
97         """
98         print(message, end="\n\n")
99         if error is not None:
100             print("-- Technical Error Message -- ")
101             print(error, error.__doc__, type(error), sep='\n')
102
```

Figure 2: Example of output_error_messages

The next two functions are output_menu and input_menu choice. Together they are used to display the menu and allow the user to navigate the menu (Figure 3).

```

102
103     @staticmethod 1 usage
104     def output_menu(menu: str):
105         """ This function displays the menu of choices to the user
106
107         ChangeLog: (Who, When, What)
108         Ryan Stoddard,03/03/2026,Created function
109         """
110         print()
111         print(menu)
112         print() # Adding extra space to make it look nicer.
113
114     @staticmethod 1 usage
115     def input_menu_choice():
116         """ This function gets a menu choice from the user
117
118         ChangeLog: (Who, When, What)
119         Ryan Stoddard,03/03/2026,Created function
120         """
121         choice = "0"
122         try:
123             choice = input("Enter your menu choice number: ")
124             if choice not in ("1", "2", "3", "4"): # Note these are strings
125                 raise Exception("Please, choose only 1, 2, 3, or 4")
126         except Exception as e:
127             IO.output_error_messages(e.__str__()) # Not passing the exception object to avoid the technical message
128         return choice
129

```

Figure 3: Example of the Menu Functions

The fourth function in this class is the `output_student_courses` function. I was careful to include the two f-strings within a print command technique I learned from the feedback on my previous assignment (Figure 4).

```

130     @staticmethod 2 usages
131     def output_student_courses(student_data: list):
132         """ This function displays all student name and course information to the user
133
134         ChangeLog: (Who, When, What)
135         Ryan Stoddard,03/03/2026,Created function
136         """
137
138         print("-" * 50)
139         for student in student_data:
140             print(f'Student {student["FirstName"]} '
141                   f'{student["LastName"]} is enrolled in {student["CourseName"]}')
142         print("-" * 50)
143

```

Figure 4: Example of the `output_student_courses` Function

The fifth and final function in this class is the `input_student_data` function. The structure of this code is very similar to the previous assignment. It asks for the user's input for name and course information while also checking to make sure there are no numbers in the names (Figure 5).

```

144     @staticmethod 1 usage
145     def input_student_data(student_data: list):
146         """ This function gathers student data from the user
147
148         ChangeLog: (Who, When, What)
149         Ryan Stoddard, 03/03/2026, Created function
150         """
151         try:
152             student_first_name = input("Enter the student's first name: ")
153             if not student_first_name.isalpha():
154                 raise ValueError("The first name should not contain numbers.")
155             student_last_name = input("Enter the student's last name: ")
156             if not student_last_name.isalpha():
157                 raise ValueError("The last name should not contain numbers.")
158             course_name = input("Please enter the name of the course: ")
159
160             student = {"FirstName": student_first_name,
161                       "LastName": student_last_name,
162                       "CourseName": course_name}
163             student_data.append(student)
164
165             print(f"You have registered {student_first_name} {student_last_name} for {course_name}.")
166         except ValueError as e:
167             IO.output_error_messages(message="That value is not the correct type of data!", e)
168         except Exception as e:
169             IO.output_error_messages(message="There was a non-specific error!", e)
170         return student_data

```

Figure 5: Example of input_student_data Function

Main Body

By far the coolest part of this whole assignment is how simple and streamlined the main body looks (Figure 6).

```

174     # Beginning of the main body of this script
175     students = FileProcessor.read_data_from_file(file_name=FILE_NAME, student_data=students)
176
177     # Repeat the following tasks
178     while True:
179         IO.output_menu(menu=MENU) #Displays the menu
180
181         menu_choice = IO.input_menu_choice() #Takes user input for menu choice
182
183         if menu_choice == "1": # Get new data (and display the change)
184             students = IO.input_student_data(student_data=students)
185             IO.output_student_courses(student_data=students)
186             continue
187
188         elif menu_choice == "2": # Displays current data
189             IO.output_student_courses(student_data=students)
190             continue
191
192         elif menu_choice == "3": # Saves data to external json file
193             FileProcessor.write_data_to_file(file_name=FILE_NAME, student_data=students)
194             continue
195
196         elif menu_choice == "4": # Ends the program
197             break # out of the while loop

```

Figure 6: Example of Main Body

It reads the data from the external JSON file, starts the while True loop to display the menu and calls all of the various functions we created as needed depending on the menu choice. It does all this while using 50 less lines of code than our previous assignment's "main body". I suppose you could argue it took 145 lines of code to set that up... but it is still pretty amazing.

Summary

The "SoC, Classes, and Functions" assignment has introduced several new concepts. We now are beginning to organize our code in a way that seems like it could scale to large complex projects in the work environment. We have the capability to create our own functions that utilize parameters, cutting down on the number of global variables. We have the capability to group together these functions into classes for some truly impressive organization and structure. And best of all, this modularity streamlines our main body of code to the point that it almost reads like English.